Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Computing
Data 2026/3/30 Class AI 1
Name Weijie Xu ID 202383910029

一、Experimental purpose

(1) To understand the fundamental concepts of containerization and lightweight service deployment. Gain a basic understanding of container-based virtualization, including the principles of Docker, image management, container orchestration, and the workflow of deploying services in an isolated environment.
(2) To acquire practical skills in deploying and managing Git services using Docker. Learn how to install and configure Gitea in a containerized environment, including service initialization, user management, and repository operations, and develop hands-on experience in deploying self-hosted version control platforms.

二、Experimental Contents

- Install Docker Desktop and verify the functionality of the container environment through basic commands and test containers.
- Deploy a Gitea service using Docker Compose, configure essential parameters such as ports, storage volumes, and service settings, and complete the initialization of the web-based interface.
- Create and manage user accounts in Gitea, and establish a new repository for version control operations.
- Enable and configure Git LFS support in Gitea, including setting the LFS service and storage path.
- Initialize a local Git repository, configure Git LFS tracking rules, and associate the repository with the Gitea remote server.
- Generate and manage a large file exceeding 1GB, and upload it to the Gitea repository using Git LFS to validate large file handling capability.
- Verify the correctness of LFS functionality by observing the upload process and repository contents, ensuring that large files are stored and retrieved properly.
- Explore repository operations such as cloning, committing, pushing, and pulling, and evaluate system behavior when handling large-scale data.

三、Detailed steps of the experiment

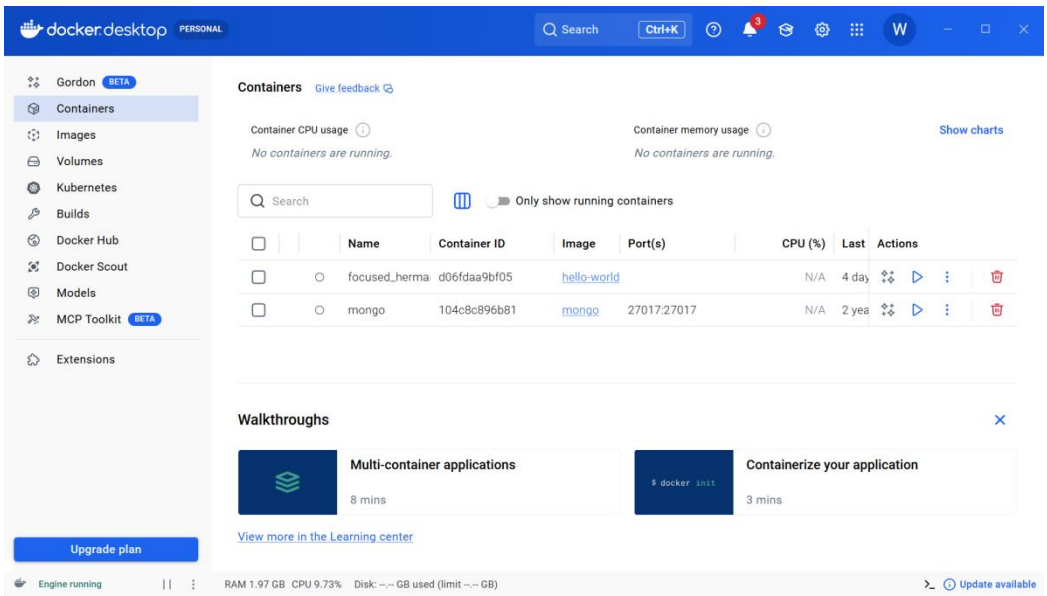
WSL environment has been successfully installed with Ubuntu 22.04 as the default distribution, and Docker has been correctly installed and is running with version 29.3.0.

```
PS C:\Users\nxdxwj\Desktop> wsl --status
默认分发: Ubuntu-22.04
默认版本: 2
PS C:\Users\nxdxwj\Desktop> docker --version
Docker version 29.3.0, build 5927d80
```

Docker is properly installed and running (version 29.3.0), and Docker Compose is also available with version v5.1.0, indicating that the container environment is ready for use.

```
PS C:\Users\nxdxwj\Desktop> docker --version
Docker version 29.3.0, build 5927d80
PS C:\Users\nxdxwj\Desktop> docker compose version
Docker Compose version v5.1.0
```

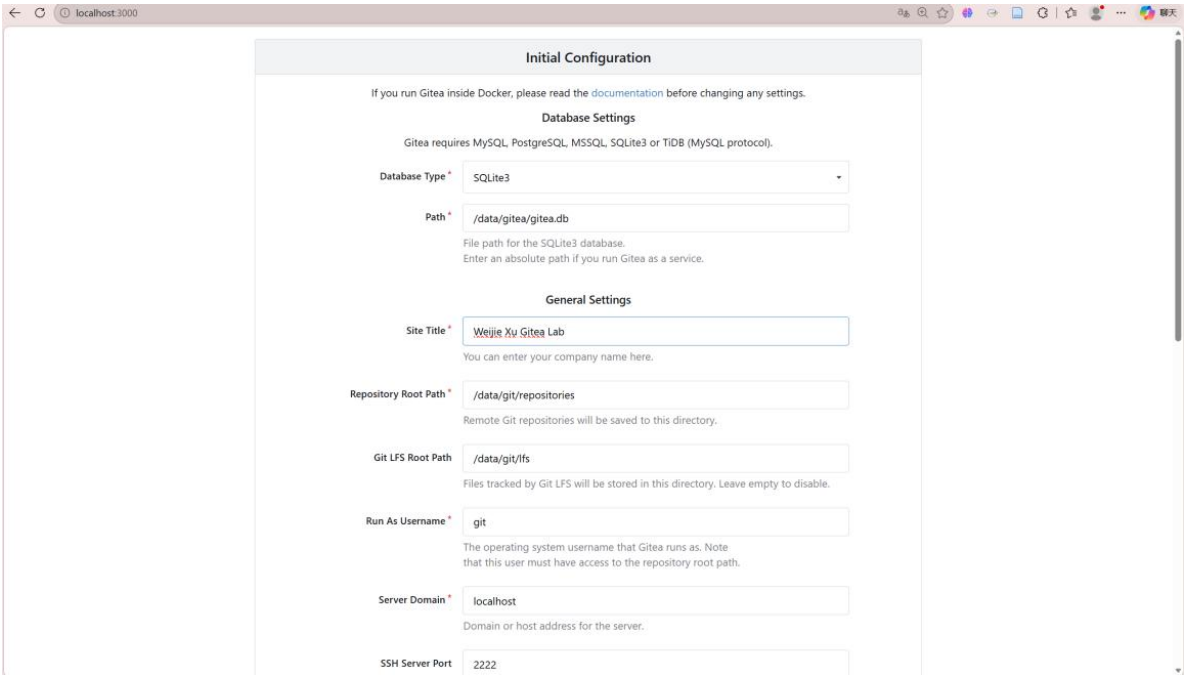
Docker Desktop interface running successfully. It displays the container management dashboard, where existing containers such as hello-world and mongo are listed, indicating that Docker is properly installed and ready for container operations.



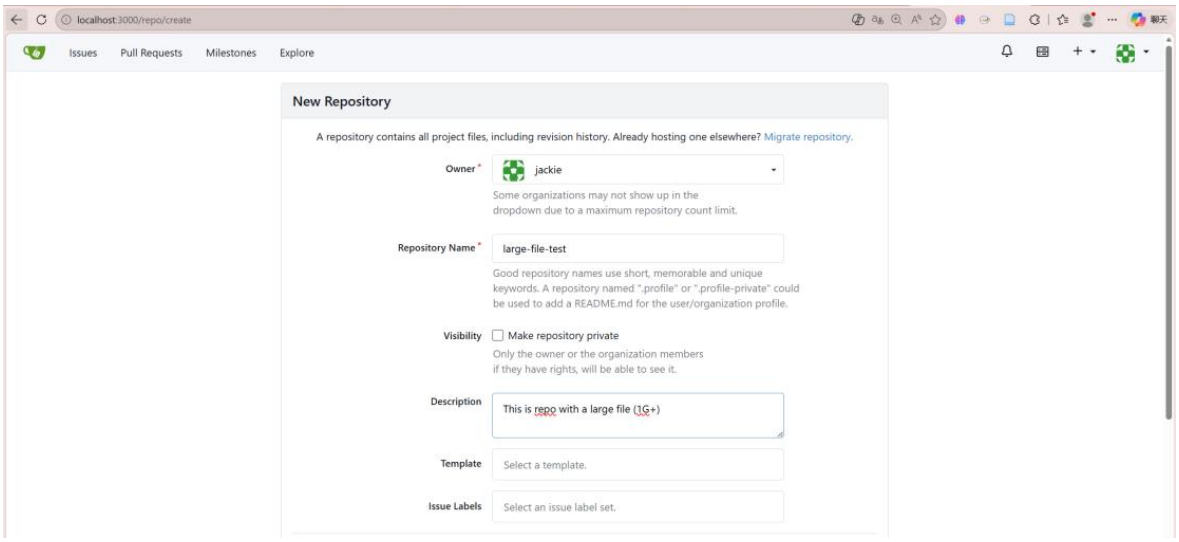
Gitea service has been successfully deployed using Docker Compose. The container is running properly, and the logs indicate that Gitea has started without errors and is ready to accept connections.

```
PS D:\cloud_lab\gitea-lab> docker compose up -d
[+] up 1/1
 ✓ Container gitea Running                                                                                                    0.0s
PS D:\cloud_lab\gitea-lab> docker compose ps
NAME      IMAGE                COMMAND                  SERVICE   CREATED         STATUS         PORTS
gitea     gitea/gitea:latest   "/usr/bin/entrypoint…"   server    3 minutes ago   Up 2 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:2222->22/tcp, [::]:2222->22/tcp
PS D:\cloud_lab\gitea-lab> docker compose logs -f
gitea  | Generating /data/ssh/ssh_host_ed25519_key...
gitea  | 2026/03/30 09:09:01 .../environment-to-ini.go:104:runEnvironmentToIni() [I] Settings saved to: "/data/gitea/conf/app.ini"
gitea  | Generating /data/ssh/ssh_host_rsa_key...
gitea  | Generating /data/ssh/ssh_host_ecdsa_key...
gitea  | Server listening on :: port 22.
gitea  | Server listening on 0.0.0.0 port 22.
gitea  | 2026/03/30 09:09:02 cmd/web.go:266:runWeb() [I] Starting Gitea on PID: 16
gitea  | 2026/03/30 09:09:02 cmd/web.go:114:showWebStartupMessage() [I] Gitea version: 1.25.5 built with GNU Make 4.4.1, go1.25.8 : bindata, timetzdata, sqlite, sqlite_unlock_notify
gitea  | 2026/03/30 09:09:02 cmd/web.go:115:showWebStartupMessage() [I] * RunMode: prod
gitea  | 2026/03/30 09:09:02 cmd/web.go:116:showWebStartupMessage() [I] * AppPath: /usr/local/bin/gitea
gitea  | 2026/03/30 09:09:02 cmd/web.go:117:showWebStartupMessage() [I] * WorkPath: /data/gitea
gitea  | 2026/03/30 09:09:02 cmd/web.go:118:showWebStartupMessage() [I] * CustomPath: /data/gitea
```

The initial configuration page of Gitea accessed via the browser. It indicates that the Gitea service is successfully running, and the system is ready for setup, including database configuration and basic server settings.



The creation of a new repository in Gitea. A repository named large-file-test is being set up, which will be used to store and manage a large file (1GB+) for testing Git LFS functionality.



Gitea configuration file (app.ini) inside the Docker container. It confirms that Git LFS support is enabled (LFS_START_SERVER = true), which allows the server to handle large file storage.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab$ docker exec gitea cat /data/gitea/conf/app.ini
APP_NAME = Weijie Xu Gitea Lab
RUN_MODE = prod
RUN_USER = git
WORK_PATH = /data/gitea

[repository]
ROOT = /data/git/repositories

[repository.local]
LOCAL_COPY_PATH = /data/gitea/tmp/local-repo

[repository.upload]
TEMP_PATH = /data/gitea/uploads

[server]
APP_DATA_PATH = /data/gitea
DOMAIN = localhost
SSH_DOMAIN = localhost
HTTP_PORT = 3000
ROOT_URL = http://localhost:3000/
DISABLE_SSH = false
SSH_PORT = 22
SSH_LISTEN_PORT = 22
LFS_START_SERVER = true
LFS_JWT_SECRET = rbflC0buwSZbwqpCipzrW5LOWhvmX-pCkQzf6eMjlc8
OFFLINE_MODE = true
```

Git LFS has been successfully installed and initialized. The message “Updated Git hooks” and “Git LFS initialized” indicates that the repository is now ready to track and manage large files using Git LFS.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/large-file-test$ git lfs install
Updated Git hooks.
Git LFS initialized.

ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/large-file-test$ git lfs track "*.largefile"
Tracking "*.largefile"
```

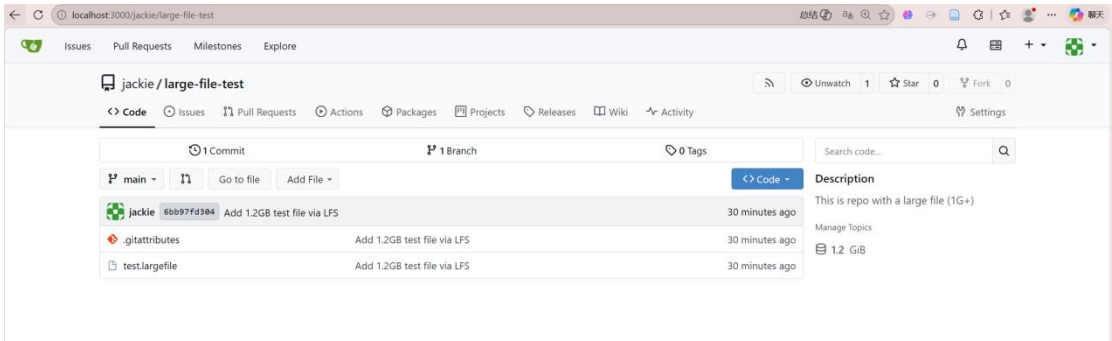
The creation of a large test file using the dd command. A file of approximately 1.2 GB has been successfully generated, which will be used to test Git LFS functionality.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/large-file-test$ dd if=/dev/zero of=test.largefile bs=1M count=1200
1200+0 records in
1200+0 records out
1258291200 bytes (1.3 GB, 1.2 GiB) copied, 3.48866 s, 361 MB/s
```

The successful push of the repository to Gitea. The message “Uploading LFS objects: 100%” confirms that the large file has been uploaded using Git LFS, and the repository has been successfully updated on the remote server.

```
ubuntu@DESKTOP-CCHACFV:~/cloud_lab/gitea-lab/large-file-test$ git push origin main
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/jackie/large-file-test.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 1.3 GB | 65 MB/s, done.
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 428 bytes | 428.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/jackie/large-file-test.git
 * [new branch]      main -> main
```

The Gitea repository after the push operation. The large file has been successfully uploaded and is stored in the repository, confirming that Git LFS is working correctly.



四、Reflection

1. This experiment provided a clear and practical understanding of containerized service deployment. By installing Docker Desktop and deploying Gitea through Docker Compose, the process of transforming theoretical concepts of cloud computing into an executable workflow became more concrete. The experience of configuring ports, volumes, and environment variables helped clarify how services are isolated, managed, and exposed in a container-based environment.

2. The experiment also highlighted the importance of network configuration and system environment in real-world deployments. Issues such as Docker engine startup failures and TLS handshake timeouts during image pulling revealed that successful deployment is not only dependent on correct commands, but also on stable networking, proper mirror configuration, and system-level dependencies like WSL.

3. In addition, working with Git LFS to manage large files deepened my understanding of the limitations of traditional Git and the necessity of specialized mechanisms for handling large-scale data. By successfully tracking and uploading a

1GB+ file, I observed how LFS separates large file storage from standard Git objects, which enhances efficiency and scalability. This experiment strengthened my overall understanding of version control in data-intensive scenarios and its relevance in cloud-based development environments.